

Mapeo con Anotaciones en Hibernate

1. Introducción al Mapeo en Hibernate

- **¿Qué es el mapeo?** Es el proceso de enlazar las clases de Java (entidades) con las tablas de una base de datos.
- **¿Por qué usar anotaciones?**
 - ✓ Son declarativas y fáciles de leer.
 - ✓ Reducen la necesidad de archivos XML (.hbm.xml).
 - ✓ Integran metadatos directamente en el código.

2. Requisitos Previos

- Framework Hibernate configurado.
- Base de datos lista para las pruebas.
- Dependencias como hibernate-core y javax.persistence (o jakarta.persistence en versiones más nuevas).

Framework configurado: Archivo Hibernate.cfg.xml

```
<hibernate-configuration>
  <session-factory>
    <!-- Configuración de conexión a la base de datos -->
    <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/Hibernate_Escuela</property>
    <property name="hibernate.connection.username">root</property>
    <property name="hibernate.connection.password">Dam2425*</property>
    <property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
    <property name="hibernate.hbm2ddl.auto">update</property>

    <!-- Clases mapeadas -->
    <mapping class="com.exampleCurso"/>
    <mapping class="com.exampleEstudiante"/>
  </session-factory>
</hibernate-configuration>
```

3. Clase Básica con Mapeo. Ejemplo sencillo para mostrar cómo se mapea una entidad a una tabla.

<https://github.com/dam2425sonia/UD3-Hibernate-Escuela>

- Clase Estudiante

```
package com.example;

import javax.persistence.*;

@Entity
@Table(name = "Estudiantes") // Mapea con la tabla 'estudiantes'
public class Estudiante {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Clave primaria con autoincremento
    private int id;

    @Column(name = "nombre", nullable = false, length = 100) // Mapea la columna 'nombre'
    private String nombre;

    @ManyToOne
    @JoinColumn(name = "curso_id", referencedColumnName = "id") // Define la clave foránea hacia 'cursos'
    private Curso curso; // Relación con Curso

    // Getters y setters
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNombre() {
        return nombre;
    }
}
```

```
public void setNombre(String nombre) {  
    this.nombre = nombre;  
}  
  
public Curso getCurso() {  
    return curso;  
}  
  
public void setCurso(Curso curso) {  
    this.curso = curso;  
}  
}
```

4. Explicación de las Anotaciones

- **@Entity**: Marca la clase como una entidad persistente.
- **@Table(name = "nombre_tabla")**: Opcional. Si no se especifica, el nombre de la tabla será igual al nombre de la clase.
- **@Id**: Define el atributo que actúa como clave primaria.
- Otros ejemplos útiles:
 - ✓ **@Column(name = "nombre_columna")**: Para personalizar el nombre de la columna.
 - ✓ **@GeneratedValue(strategy = GenerationType.IDENTITY)**: Para indicar que el valor se genera automáticamente en la base de datos.

5. Relaciones entre Entidades

- **@OneToOne**
- **@OneToMany**
- **@ManyToOne**
- **@ManyToMany**

Ser el propietario en las anotaciones de Hibernate significa que un lado de la relación es responsable de mantener y persistir la asociación en la base de datos, principalmente a través de la clave foránea. En otras palabras, el propietario es el lado que Hibernate utiliza para actualizar y gestionar la relación en la base de datos.

Cuando trabajamos con relaciones en Hibernate (@OneToOne, @OneToMany, @ManyToOne, @ManyToMany), solo uno de los lados de la relación es el propietario. El otro lado es el mapeo inverso (o lado no propietario).

¿Qué responsabilidades tiene el propietario?

- Definir y almacenar la clave foránea: El propietario es el lado donde se encuentra físicamente la columna de clave foránea en la base de datos.
- Controlar los cambios en la relación: Solo las modificaciones realizadas en el lado propietario se reflejan correctamente en la base de datos.

El lado inverso simplemente describe la relación pero no gestiona los datos en la base de datos.

- **@OneToMany**: En una relación @OneToMany, el lado propietario es el que tiene la clave foránea en la base de datos. Este suele estar en el lado de la relación @ManyToOne. Aunque @OneToMany define la relación, el propietario en términos de base de datos y persistencia será el lado que contiene la columna con la clave foránea, que generalmente es el lado @ManyToOne
- **@OneToOne**: El propietario es el lado que define la clave foránea con @JoinColumn.
- **@ManyToMany**: En una relación @ManyToMany, el propietario es el lado que define la tabla intermedia con la anotación @JoinTable.

¿Qué implica ser el propietario en Hibernate?

- Control de la clave foránea:
 - ✓ El propietario es quien define y almacena la clave foránea o la tabla de unión en la base de datos.
 - ✓ Es el lado de la relación que Hibernate utiliza para realizar insert, update y delete en la clave foránea o la relación.
- Sincronización de la base de datos:
 - ✓ Solo los cambios realizados en el lado propietario se reflejan en la base de datos.
 - ✓ Si haces cambios en el lado inverso (el lado no propietario), estos no se persisten a menos que el lado propietario también esté sincronizado.
- Uso de anotaciones específicas:
 - ✓ El lado propietario es el que contiene las anotaciones relacionadas con la clave foránea o la relación, como:
 - @JoinColumn (en relaciones @ManyToOne o @OneToOne)
 - @JoinTable (en relaciones @ManyToMany).
- Configuración de relaciones bidireccionales:
 - ✓ En relaciones bidireccionales, el lado inverso debe usar la anotación mappedBy para referenciar al atributo en el lado propietario. Esto le indica a Hibernate que no debe gestionar la relación en el lado inverso.
- Persistencia y gestión:
 - ✓ Si persistes o modificas la relación en el lado no propietario, Hibernate no actualizará automáticamente la base de datos. Debes asegurarte de realizar los cambios desde el lado propietario para que Hibernate los tome en cuenta.

Ejemplo: Relación @OneToMany

- Clase Curso

```
package com.example;

import java.util.List;
import javax.persistence.*;

@Entity
@Table(name = "Cursos") // Mapea con la tabla 'cursos'
public class Curso {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // Clave primaria con autoincremento
    private int id;

    @Column(name = "nombre", nullable = false, length = 50) // Mapea la columna 'nombre'
    private String nombre;

    @OneToMany(mappedBy = "curso", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<Estudiante> estudiantes; // Relación con Estudiantes

    // Getters y setters
    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
```

```
        this.nombre = nombre;
    }

    public List<Estudiante> getEstudiantes() {
        return estudiantes;
    }

    public void setEstudiantes(List<Estudiante> estudiantes) {
        this.estudiantes = estudiantes;
    }
}
```

Ejemplo: Relación @ManyToOne

- Relación en la clase Estudiante:

```
@ManyToOne
@JoinColumn(name = "curso_id", referencedColumnName = "id") // Define la clave foránea hacia 'cursos'
private Curso curso; // Relación con Curso
```

6. Ventajas del Mapeo con Anotaciones

- Menos propenso a errores que los archivos XML.
- Código más limpio y legible.
- Mejora la productividad en el desarrollo.

7. Actividad Práctica

1. Crea estas clases con anotaciones básicas: `@Entity`, `@Id`, y `@Table`.
2. Fíjate en las relaciones creadas, como **@OneToMany** o **@ManyToOne** y entiéndelas.
3. Crea un programa para guardar y consultar entidades con Hibernate, mostrando cómo interactúan las clases con la base de datos. A continuación se muestran ejemplos de cómo realizar el guardado y la consulta. Pruébalos.

Ejemplo de Guardado:

```
package com.example;

import java.util.Arrays;
import org.hibernate.Session;
import org.hibernate.Transaction;

public class Main {
    public static void main(String[] args) {
        Session session = HibernateUtil.getSessionFactory().openSession();
        Transaction transaction = session.beginTransaction();

        // Crear curso
        Curso curso = new Curso();
        curso.setNombre("Matemáticas");

        // Crear estudiantes
        Estudiante estudiante1 = new Estudiante();
        estudiante1.setNombre("Juan Pérez");
        estudiante1.setCurso(curso);
        Estudiante estudiante2 = new Estudiante();
        estudiante2.setNombre("Ana Gómez");
        estudiante2.setCurso(curso);

        // Asociar estudiantes al curso
        // Lista de nombres de ejemplo
        curso.setEstudiantes(Arrays.asList(estudiante1, estudiante2));
        // Guardar curso (cascada guardará también los estudiantes)
```

```
        session.save(curso);

        transaction.commit();
        session.close();
    }
}
```

Ejemplo de Consulta:

```
package com.example;

import java.util.List;
import org.hibernate.Session;

public class Consultar {
    public static void main(String[] args) {
        Session session = HibernateUtil.getSessionFactory().openSession();

        List<Estudiante> estudiantes = session.createQuery("FROM Estudiante", Estudiante.class).list();
        for (Estudiante estudiante : estudiantes) {
            System.out.println("Estudiante: " + estudiante.getNombre() +
                ", Curso: " + (estudiante.getCurso() != null ? estudiante.getCurso().getNombre() : "Sin curso"));
        }

        session.close();
    }
}
```

8. Preguntas:

- ¿Qué pasa si olvidamos usar @Id?
- ¿Cómo afecta la anotación @Column si no la definimos explícitamente?